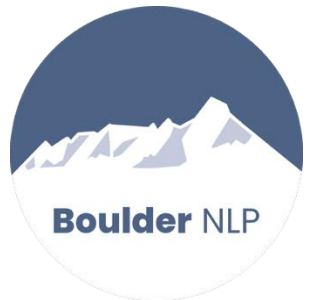


Natural Language Processing

Jim Martin -- CSCI 5832

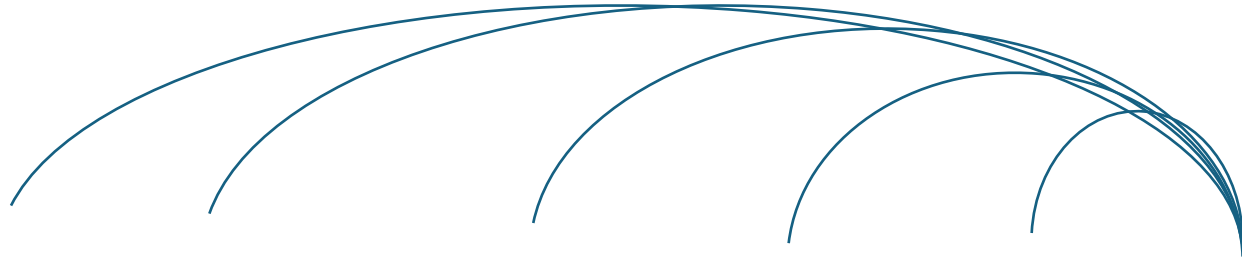
Lecture 12



Today

- Reminder: Thursday is Reading Day, no class.
- Transformers
 - Chapter 7: 7.1 to 7.5
- SemEval Tasks and Teams

Simple Attention in Sequence Processing



...horrific car accident he became an organ

$$\textit{score}(x_i, x_j) = x_i \cdot x_j$$

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\textit{score}(x_i, x_j)) \quad \forall j \leq i \\ &= \frac{\exp(\textit{score}(x_i, x_j))}{\sum_{k=1}^i \exp(\textit{score}(x_i, x_k))} \quad \forall j \leq i\end{aligned}$$

$$y_i = \sum_{j \leq i} \alpha_{ij} x_j$$

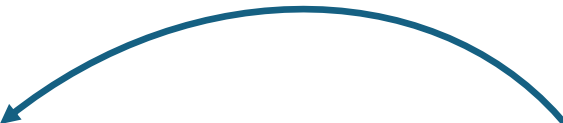
3 Roles for Embeddings in Self-Attention

1. As the current focus of attention, being compared to other words (q)
2. As a word being compared to the current focus of attention (k)
3. As the embedding used to generate an output for the current input (v)

Self-Attention in Sequence Processing

Role 1

...horrific car accident he became an organ



Accident, as the focus of attention, is being compared to horrific.

Self-Attention in Sequence Processing

Role 2

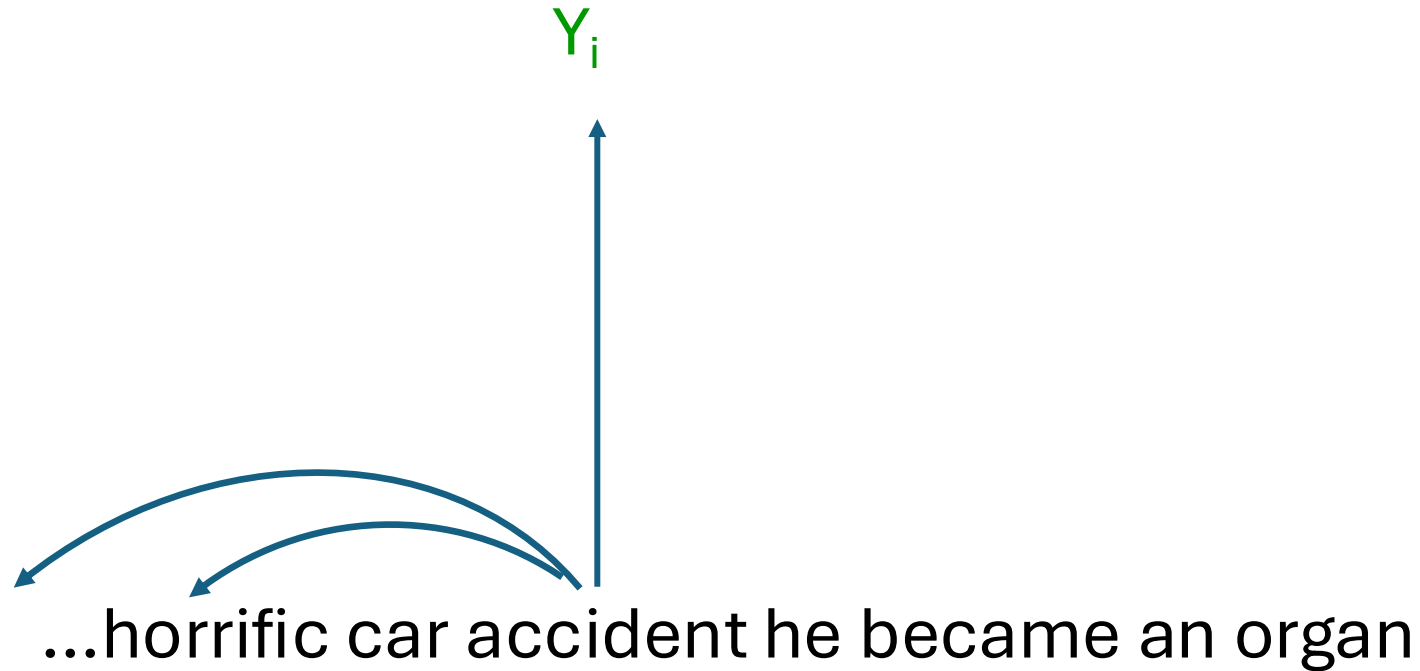
...horrific car accident he became an organ



Accident is being compared against organ, with organ as the focus of attention.

Self-Attention in Sequence Processing

Role 3



Accident is the focus of attention being used to produce an output at this timestep.

Queries, Keys and Values

To distinguish these roles, we'll introduce new embeddings and weight matrices to capture them

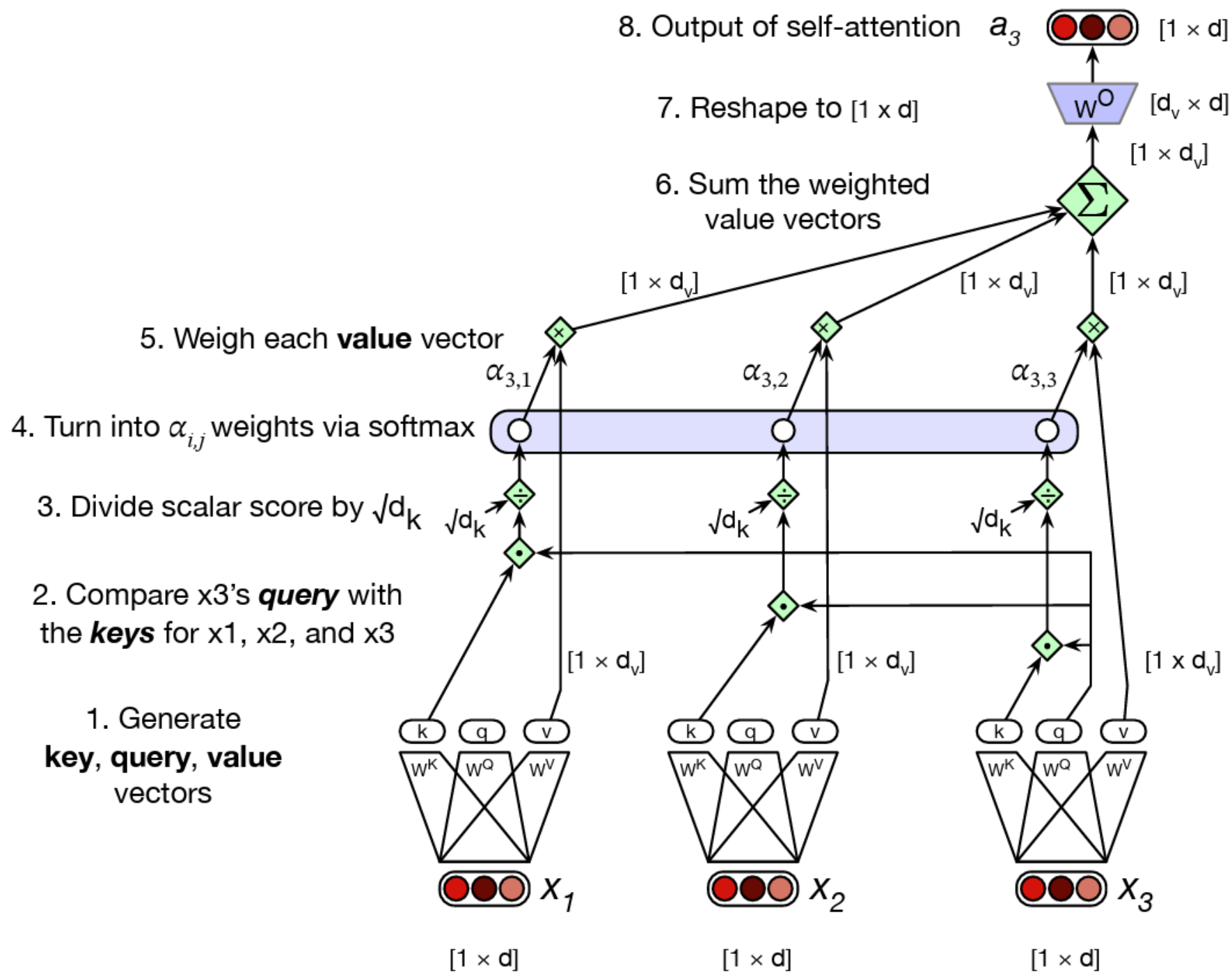
$$q_i = W^Q x_i; \quad k_i = W^K x_i; \quad v_i = W^V x_i$$

- Update our scoring function

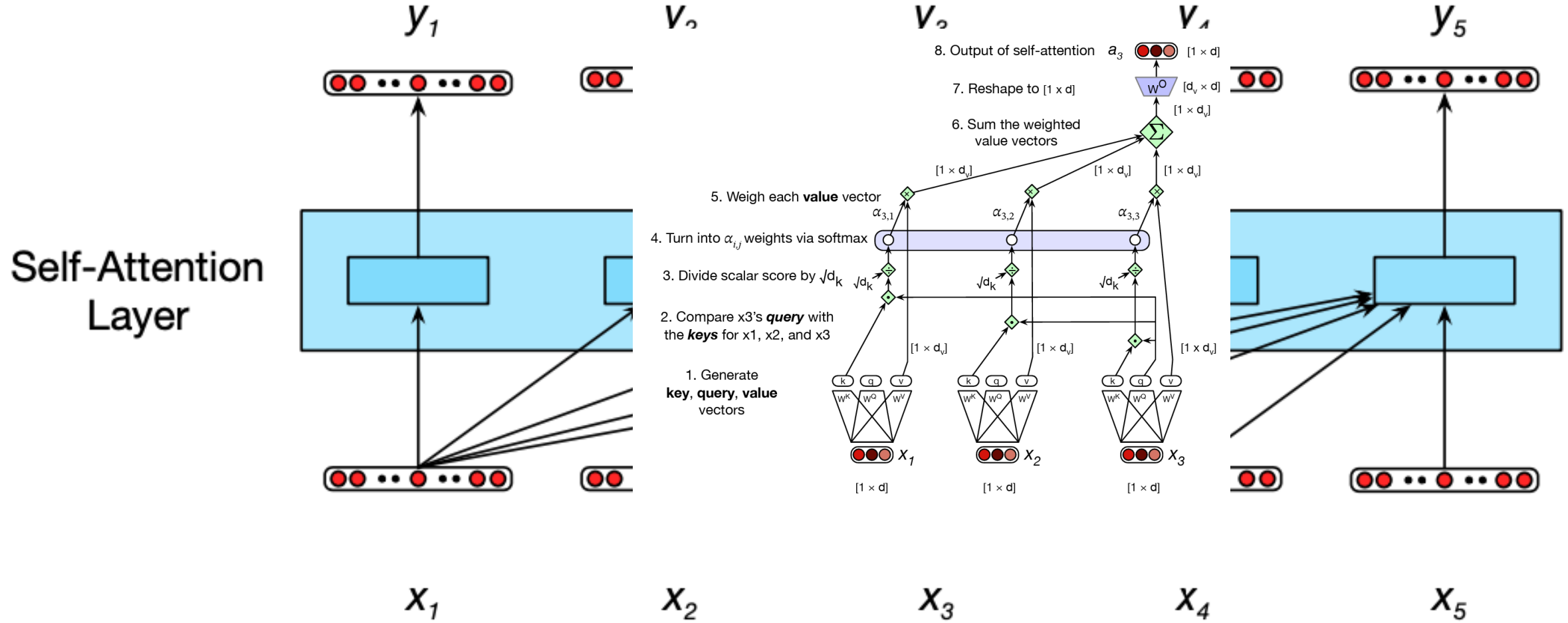
$$\text{score}(x_i, x_j) = q_i \cdot k_j$$

- Update the value calculation

$$y_i = \sum_{j \leq i} \alpha_{ij} v_j$$



Causal Self-Attention Layer



Parallel Vector Processing

Thus far the attention mechanism was described as computing a single output y_i , one item at a time. But we can easily exploit the parallel nature of the computation.

- Computing each output can be done independently of all the other outputs.
- And each output computation is based on exactly the same set of steps.

Parallel Attention

$$q_i = W^Q x_i; \quad k_i = W^K x_i; \quad v_i = W^V x_i$$

Replace

With

$$\mathbf{Q} = \mathbf{XW}^Q; \quad \mathbf{K} = \mathbf{XW}^K; \quad \mathbf{V} = \mathbf{XW}^V$$

That is, pack all the input x vectors into a matrix X and matrix multiply to produce each of the q , k and v vectors in a single step.

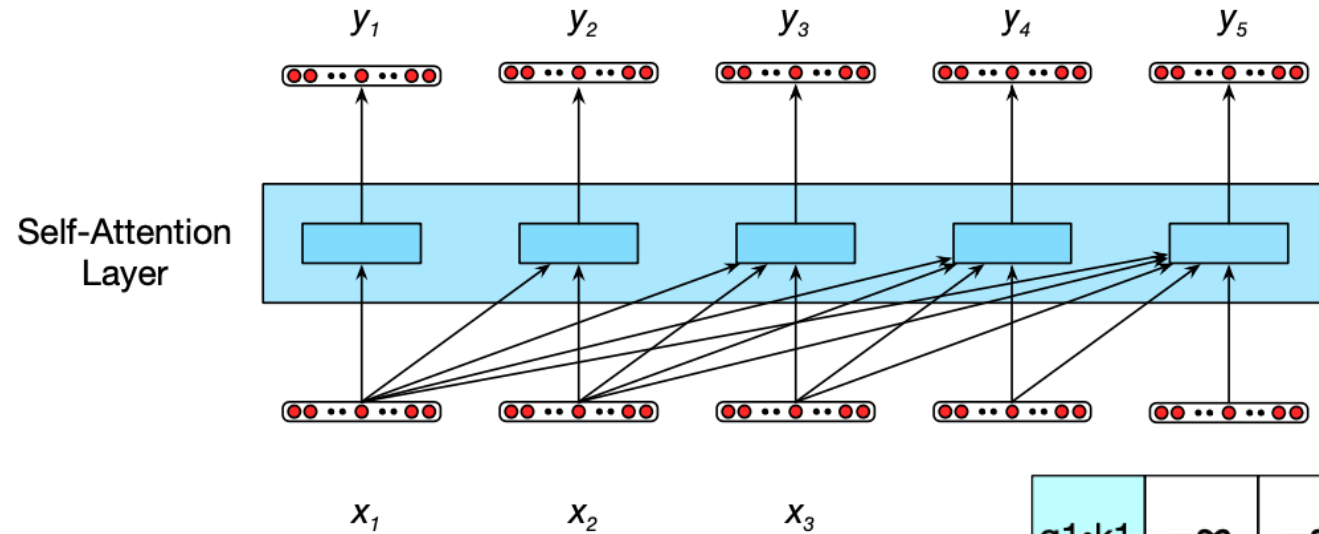
Parallel Attention (2)

- With this arrangement, we can do all the query/key dot products at once with a single matrix multiply

QK^T

N	q1•k1	q1•k2	q1•k3	q1•k4	q1•k5
	q2•k1	q2•k2	q2•k3	q2•k4	q2•k5
	q3•k1	q3•k2	q3•k3	q3•k4	q3•k5
	q4•k1	q4•k2	q4•k3	q4•k4	q4•k5
	q5•k1	q5•k2	q5•k3	q5•k4	q5•k5
N					

Not Quite Right



N

q1•k1	q1•k2	q1•k3	q1•k4	q1•k5
q2•k1	q2•k2	q2•k3	q2•k4	q2•k5
q3•k1	q3•k2	q3•k3	q3•k4	q3•k5
q4•k1	q4•k2	q4•k3	q4•k4	q4•k5
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

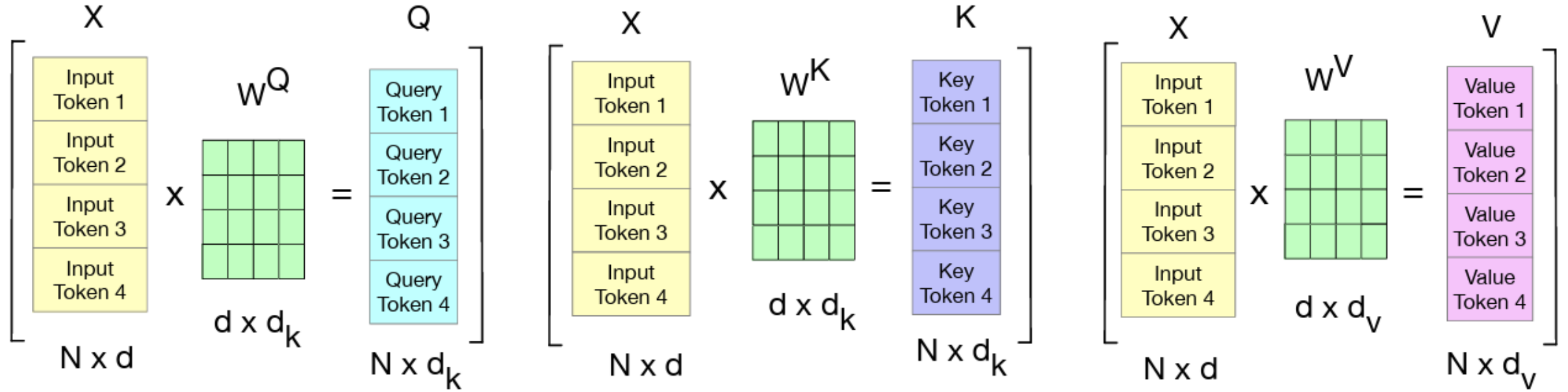
N

What does this cell represent?

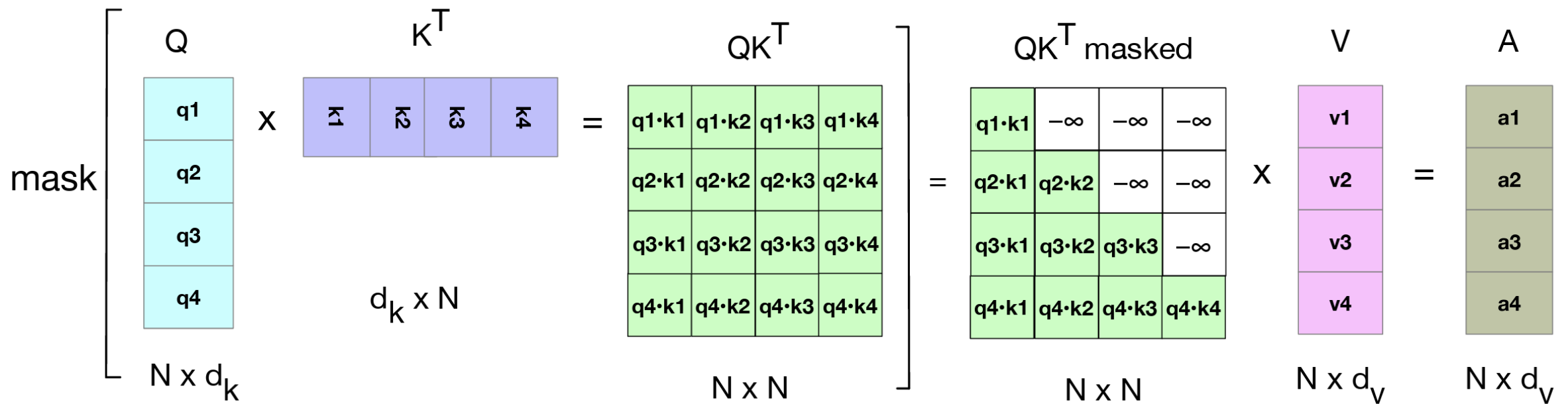
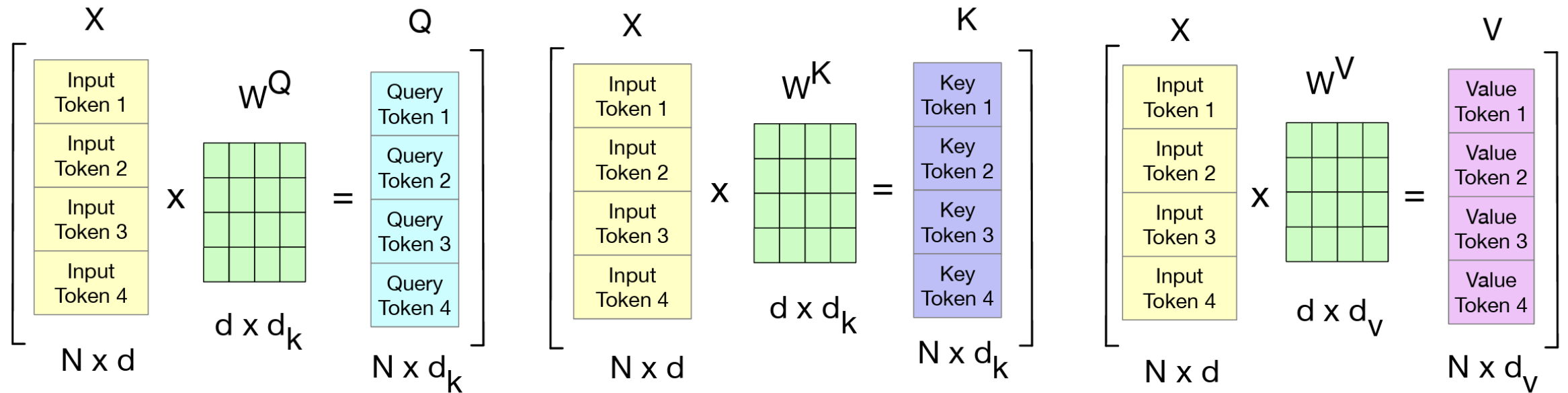
N

q1•k1	$-\infty$	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4	$-\infty$
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N



$$Q = XW^Q; \quad K = XW^K; \quad V = XW^V$$



Finally

Final output calculation

$$\textit{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax} \left(\underbrace{\frac{\mathbf{Q}\mathbf{K}^\top}{\underbrace{\sqrt{d_k}}_{\text{Scaling}}} \right)}_{\text{Softmax to produce } \alpha} \mathbf{V}$$

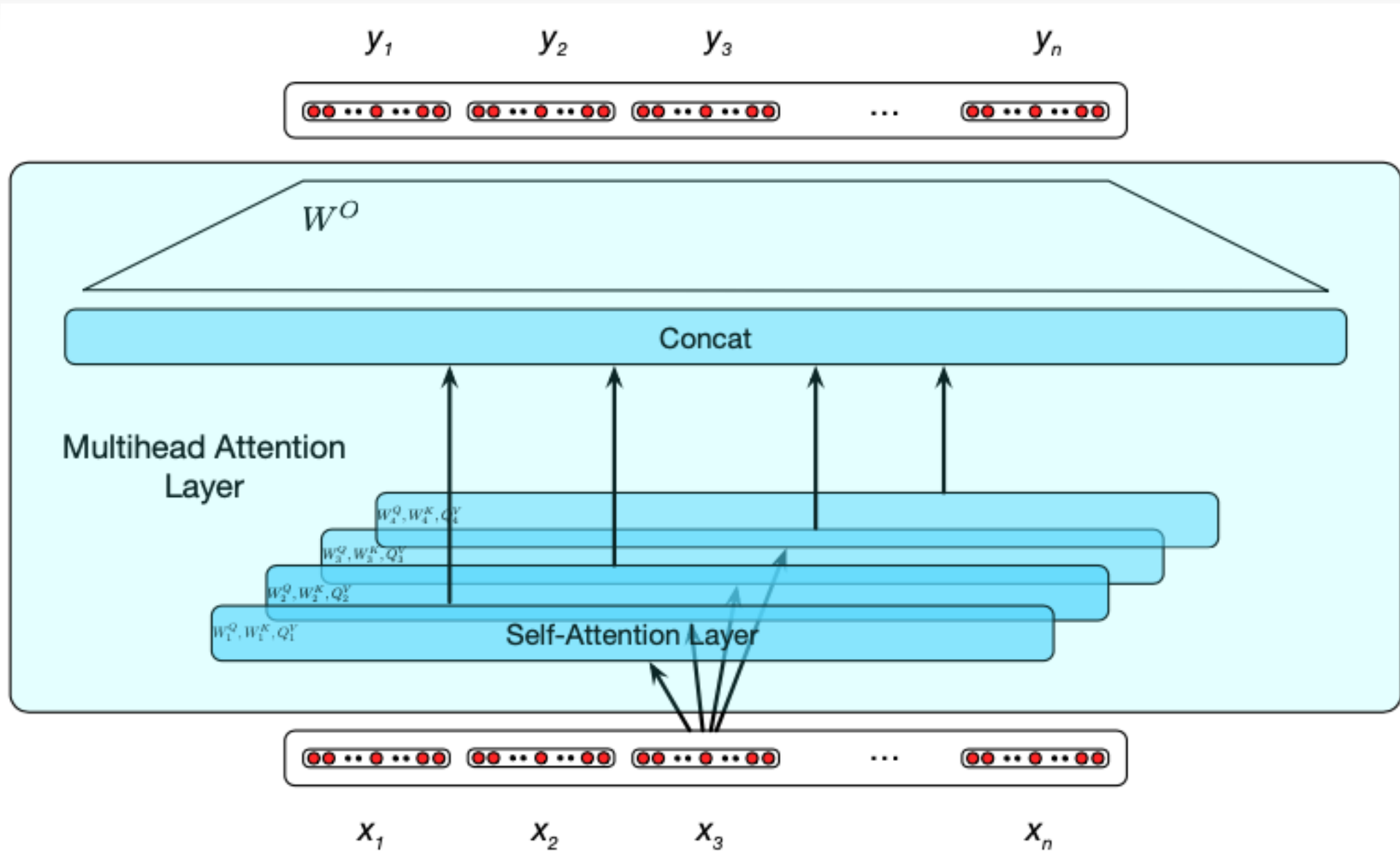
Query/Key dot products

Scaling

Multihead Attention

- We can add additional self-attention layers in parallel within a block to give the models more expressive power
 - Increase the model's ability to infer representations useful to downstream processing.
 - Each layer has its own set of weights W (key, query, value weights).

Multihead Attention



Multihead Attention

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}c}; \quad \forall c \quad 1 \leq c \leq A \quad (8.15)$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}} \quad (8.16)$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i \quad (8.17)$$

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c \quad (8.18)$$

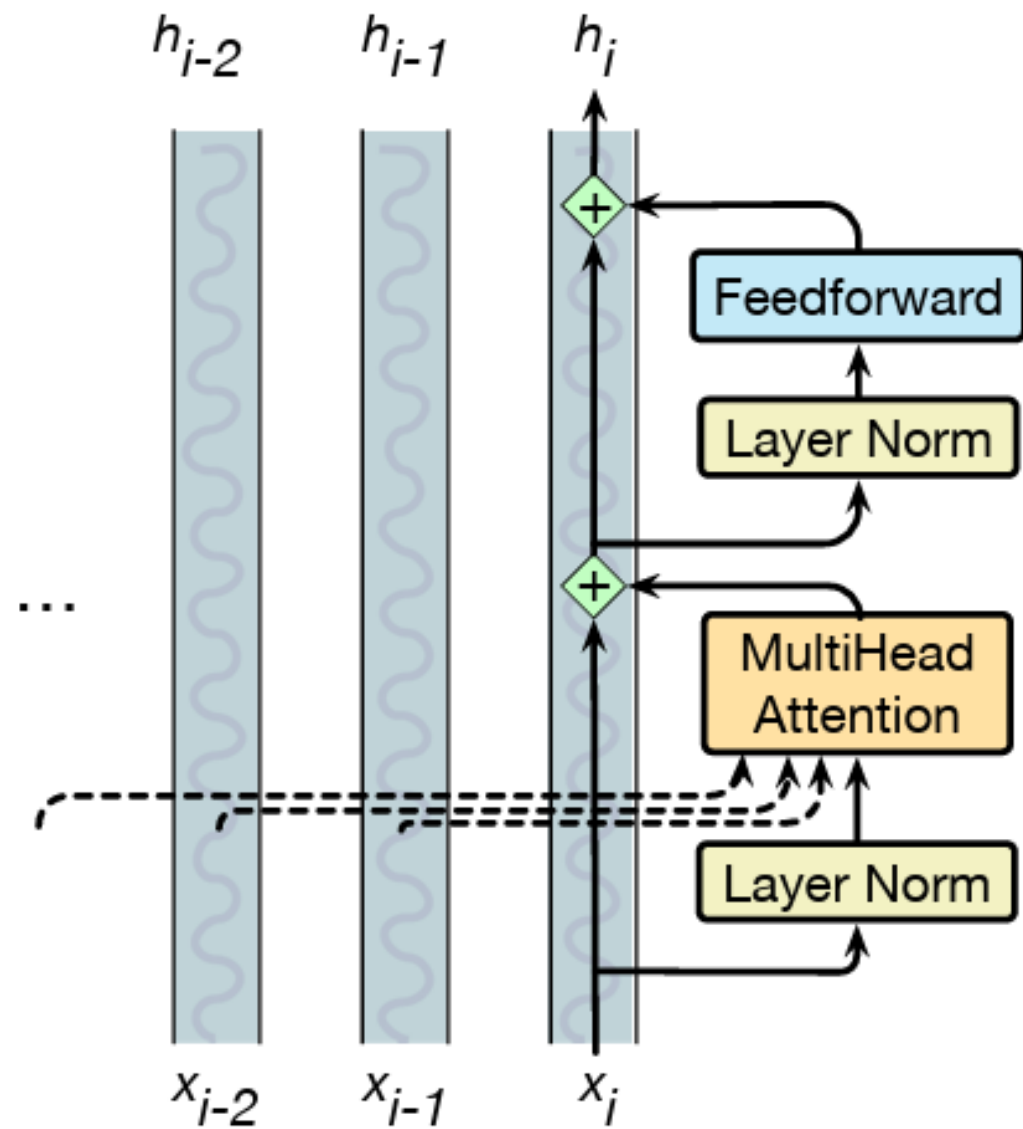
$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^A) \mathbf{W}^O \quad (8.19)$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_{i-1}]) = \mathbf{a}_i \quad (8.20)$$

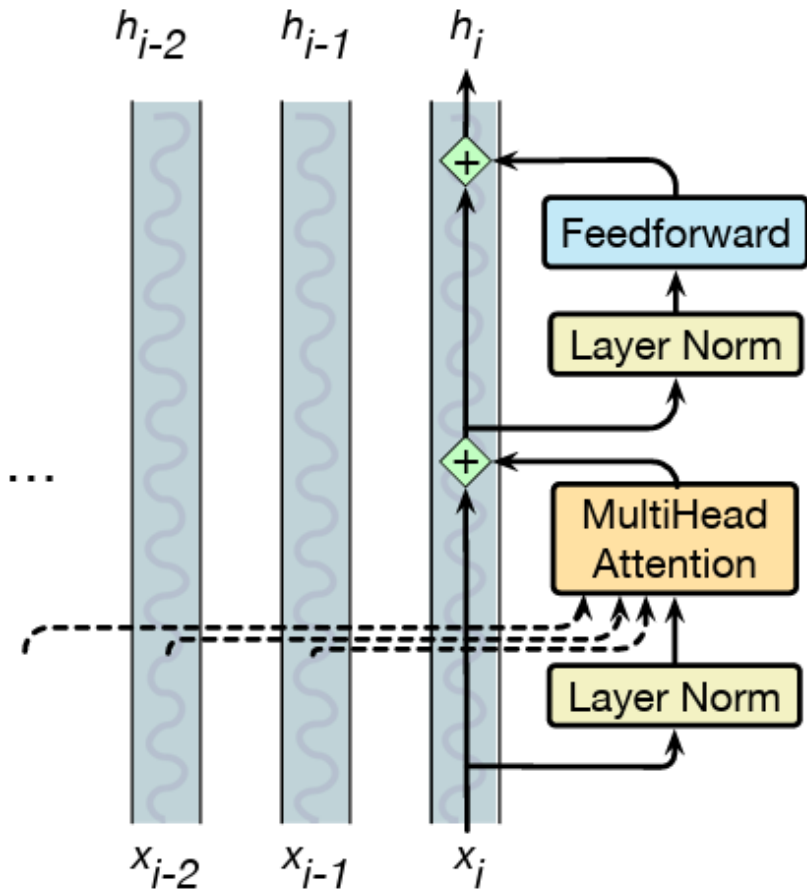
Transformer Blocks

- All of that is just for the self-attention computation (layer)
- Transformers consist of multiple self-attention layers along with additional network connections among them.
- These blocks can then be stacked as we did with RNNs
 - Vectors in vectors out

Transformer Blocks



Transformer Block/Residual Stream

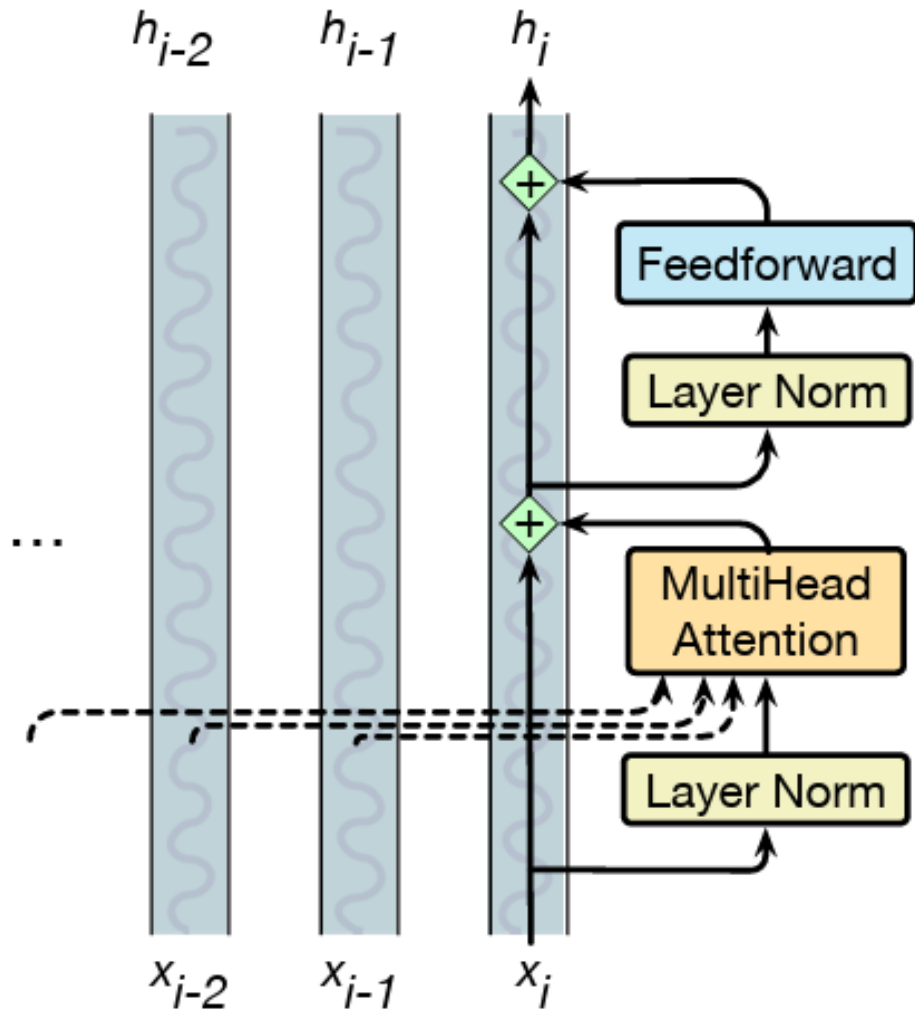


$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

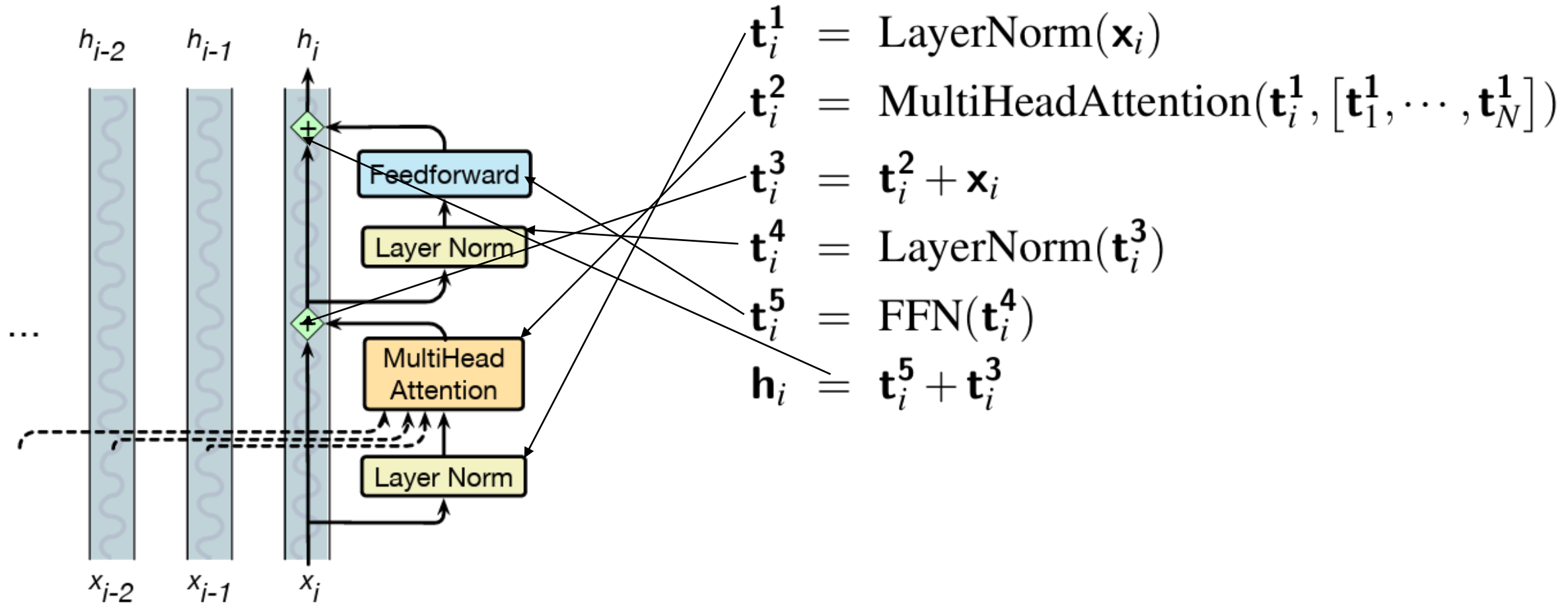
$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

Transformer Block/Residual Stream



$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$

Transformer Block



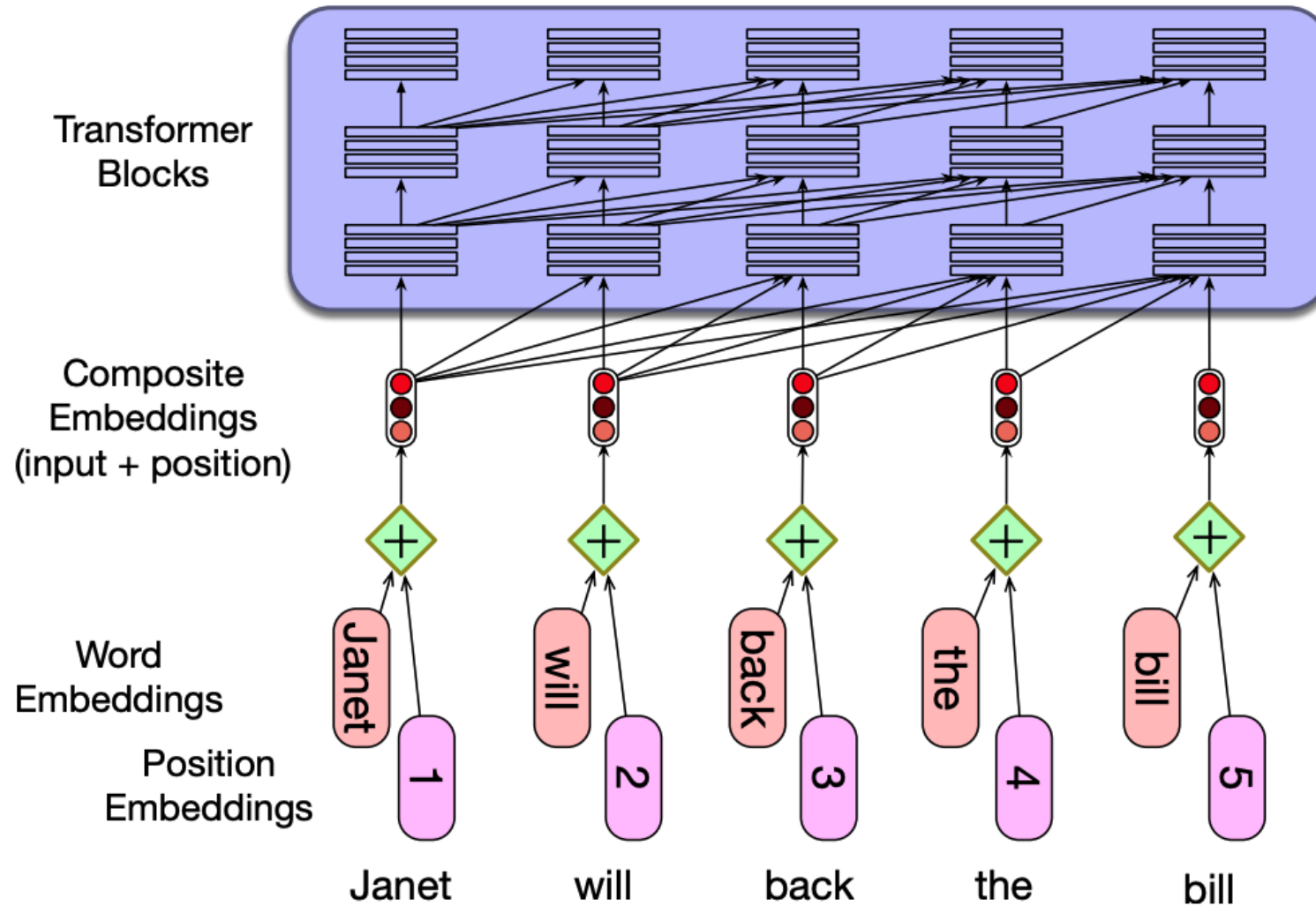
Positional Embeddings

- Recall that one motivation for the design of RNNs was to be able to treat sequences directly as sequences.
- But what we've just described with self-attention doesn't really do that. The past is an unordered bag, not a sequence. Repeated instances of the same word can't even be distinguished.

Positional Embeddings

- Positional embeddings are a way to add either exact or relative location information to the input embeddings.
- Simplest approach is to train embeddings for each possible position in a sequence. Then add the appropriate positional embedding to each input embedding.

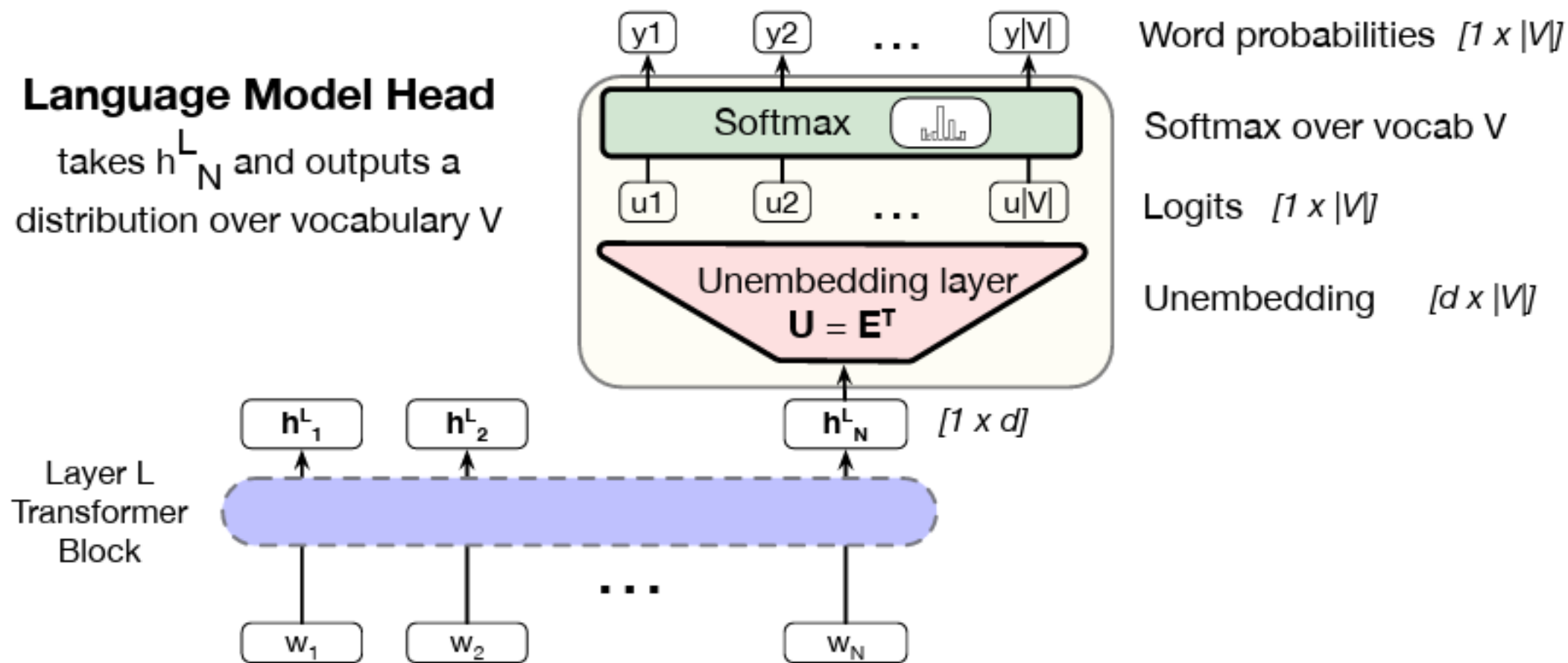
Positional Embeddings



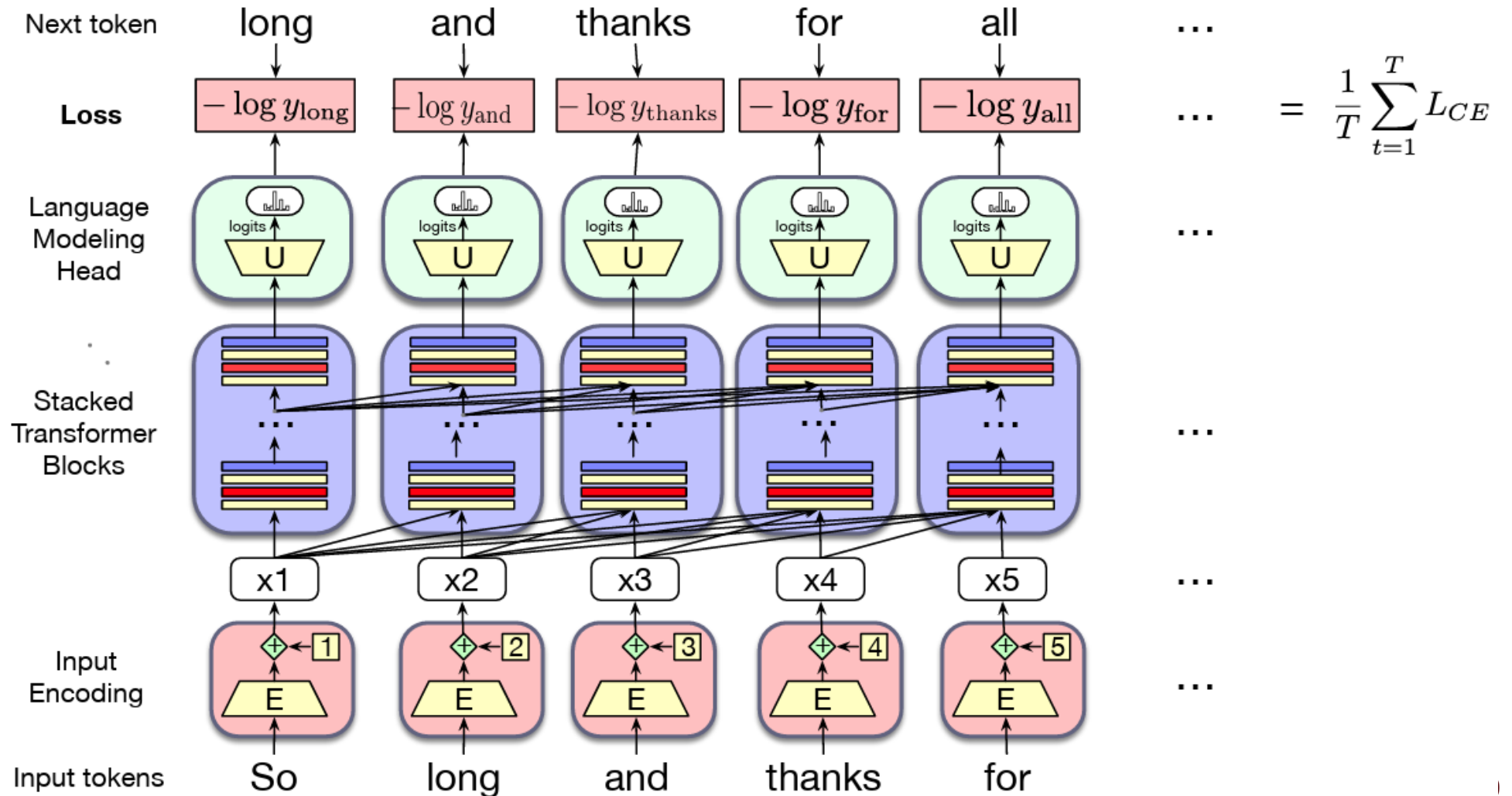
Transformer as Language Model: The Language Model Head

Language Model Head

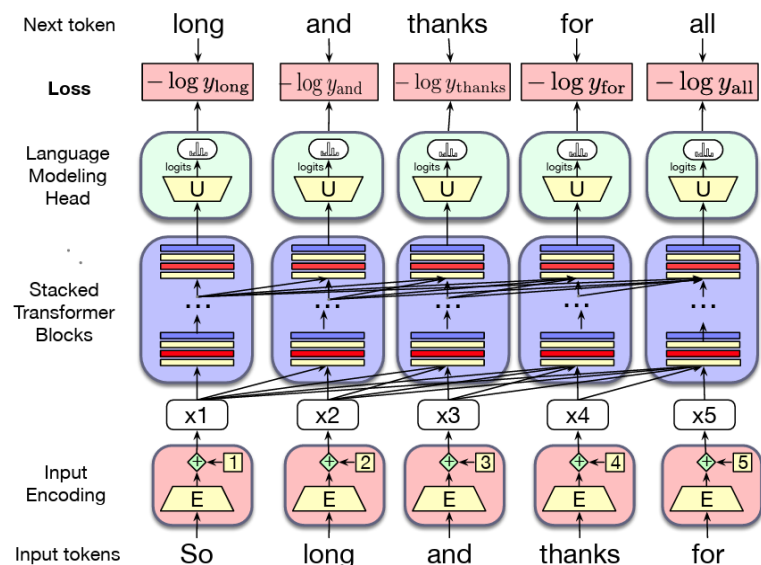
takes h_N^L and outputs a distribution over vocabulary V



Transformer Language Model Training



Transformer Language Model Training



No temporal dependence here. Given a training text, we know the right answer for each point in time. And each of those answers is independent of all the other answers (the forward computation for each time step is independent of the other time steps).

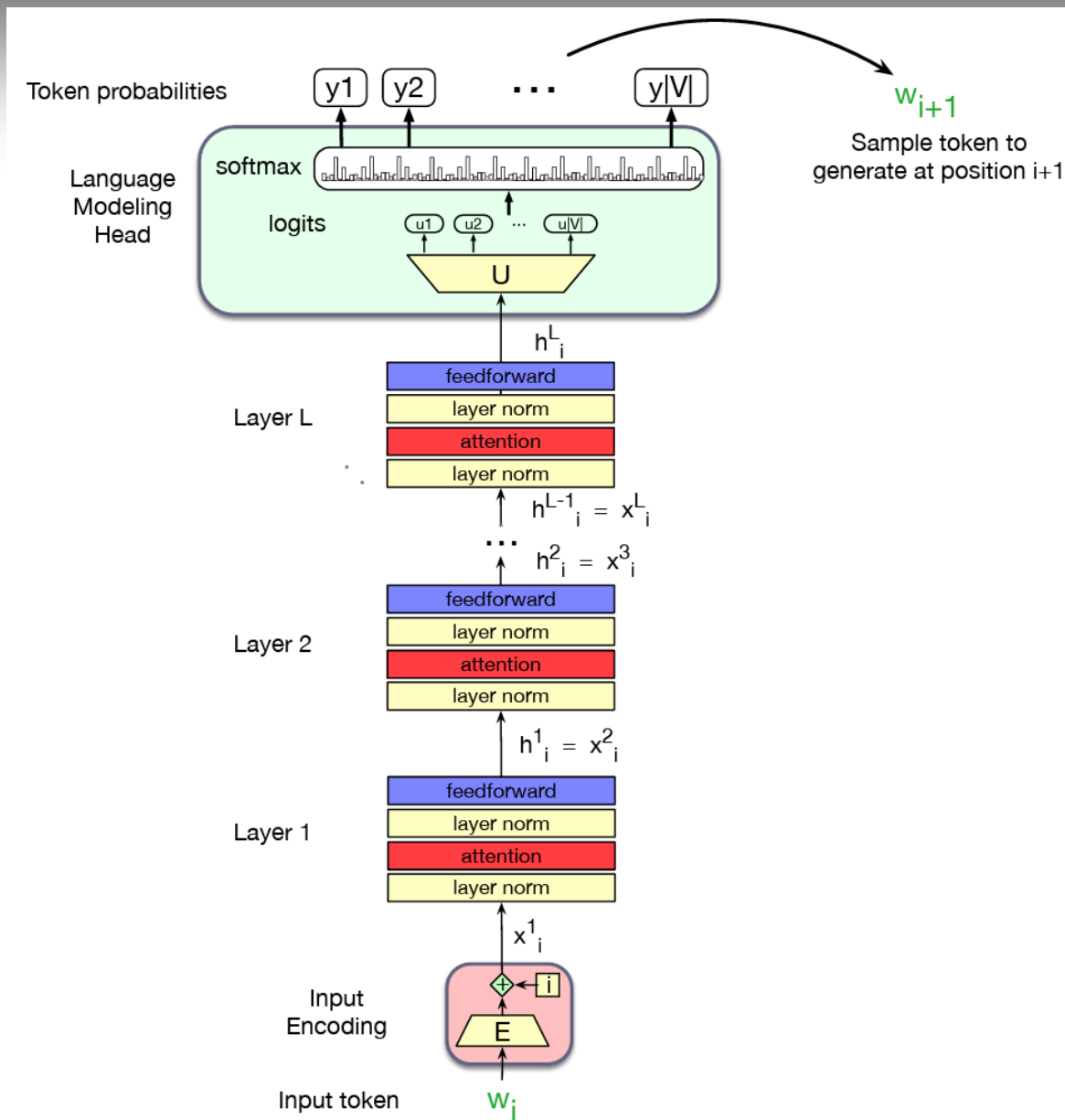
$$= \frac{1}{T} \sum_{t=1}^T L_{CE}$$

- As with other gradient-descent methods, we don't want to do a weight update on each item. We can use a minibatch-based approach. Accumulate a loss over a batch, average it, and use that for a single weight update.

- This is a huge advantage over RNN training which is inherently serial due to the recurrence.

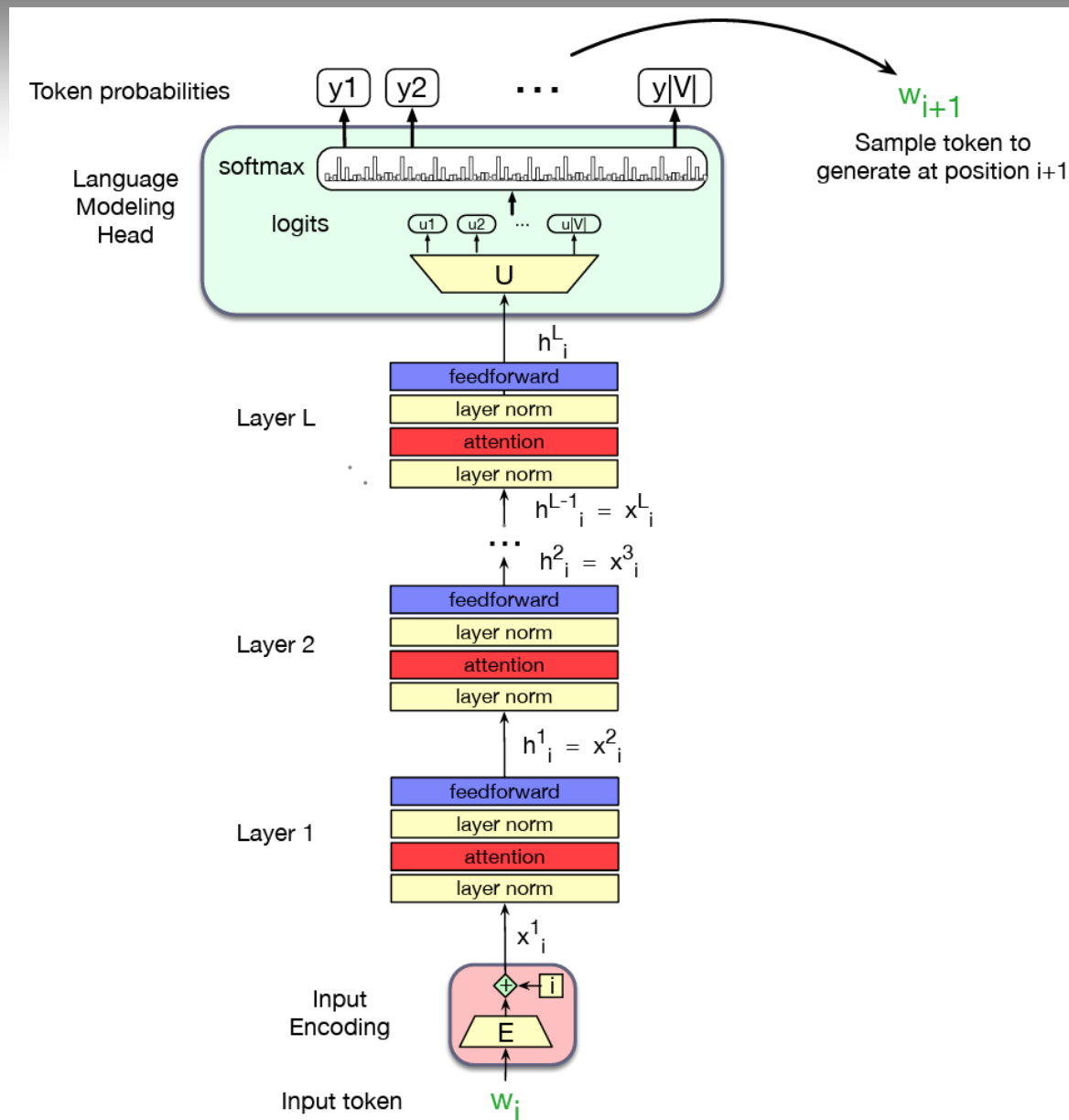
Inference (Test-time) Compute

But what about forward inference?



Inference (Test-time) Compute

Inference based on autoregressive decoding is inherently serial. You can't decode at time $i+1$ without first having sampled to get the output at time i .



Model Size Example

GPT 3 (175B)

- 50k BPE vocabulary
- Embedding layer/ model dimensionality: 12,288
 - Meaning embedding matrix E is... $(50k \times 12k) = \sim 600M$ parameters
 - As is the language modeling head U
- 96 Transformer blocks (layers)
 - 96 attention heads per block (multihead attention)
 - FF hidden layers 49k (4x the model dim)